DISSERTATION PROPOSAL PRESENTATION

CropAssist

An AI-Assisted Crop Recommendation and Farm Decision Support System Using Machine Learning

Python · FastAPI · React · PostgreSQL · scikit-learn



Presented by	[Student Name] · [BITS ID]
Supervisor	[Supervisor Name], [Designation]
Organisation	[Organisation / Mentor Name]

BITS Pilani · CSIW ZG628T Dissertation · Academic Session 2026–2027

OVERVIEW

Presentation Outline

What we will cover in the next 10–15 minutes

01 Problem and motivation

02 Proposed solution

03 Objectives and scope

04 Architecture and workflow

05 Machine-learning methods

06 Database design

07 Evaluation plan

08 Timeline and conclusion

THE PROBLEM

Why Crop Selection Is Challenging

Crop suitability depends on many factors at once

- Soil nutrient levels (N, P, K)
- Soil acidity or alkalinity (pH)
- Temperature and humidity
- Rainfall
- Local farming conditions
- Season and crop-specific needs

PROBLEM STATEMENT

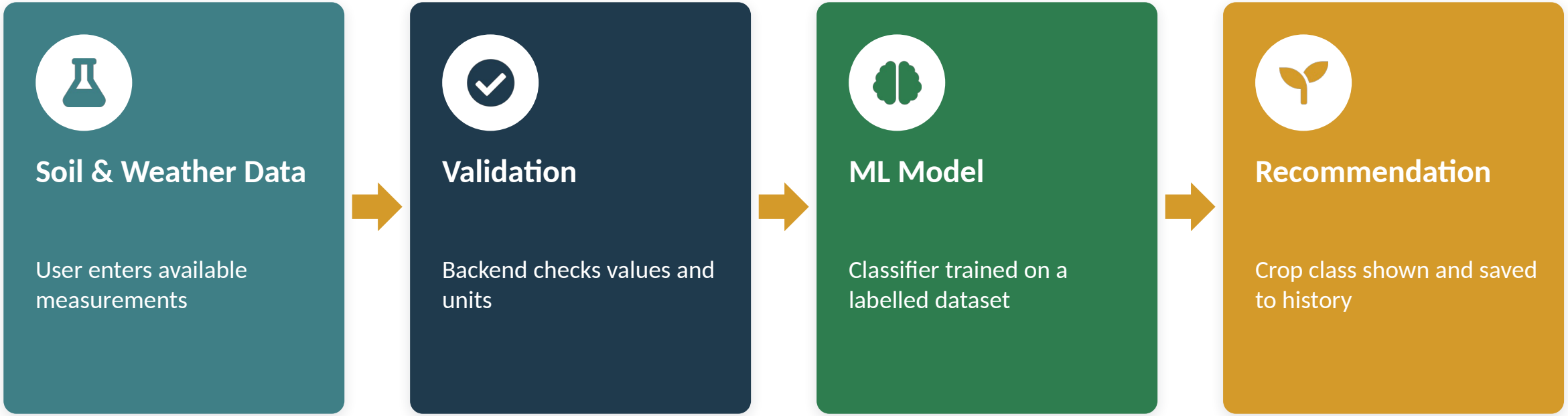
How can a web application use available soil and environmental parameters to provide model-based crop-class recommendations?

Why it matters
Weighing several measurements together is hard without decision-support information.

OUR PROPOSED SOLUTION

CropAssist in One Picture

A web application with a lightweight ML model



In one sentence: CropAssist proposes a web app where users enter soil and weather measurements and a trained machine-learning model suggests a crop class — as decision support, not a guarantee.

OBJECTIVES

What We Propose to Build

Ten objectives in three groups

Application

- Registration and login
- Farm / field records
- Input forms and results screen
- Recommendation history

AI / ML

- Dataset preparation
- Compare lightweight classifiers
- FastAPI prediction endpoint
- Model-version tracking

Evaluation

- Software testing
- ML model evaluation

RESEARCH CORE

comparing lightweight supervised classifiers and serving the selected model through a traceable web workflow.

SCOPE

What Is In — and Out — of the First Release

A smaller, well-tested system is stronger than an oversized one

In scope

- Manual parameter entry
- Selected supported crop classes
- Lightweight supervised ML
- Recommendation history
- User and admin interfaces
- Model-version traceability
- Software testing and ML evaluation

Out of scope / future work

- IoT sensors
- Satellite imagery
- Fertilizer prescriptions
- Image-based disease diagnosis
- Yield guarantees
- Universal recommendations

Recommendations are informational — users should consult qualified agricultural guidance.

SYSTEM ARCHITECTURE

How the Technologies Work Together

Proposed layered design

User — Farmer, admin or student researcher

↓

React frontend — Forms, results, history and dashboards

↓

FastAPI backend — REST APIs, validation, authentication, workflow

↓

Python ML inference — Preprocessing and prediction with the saved model (joblib)

PostgreSQL

Users, fields, recommendation inputs and outputs, model versions

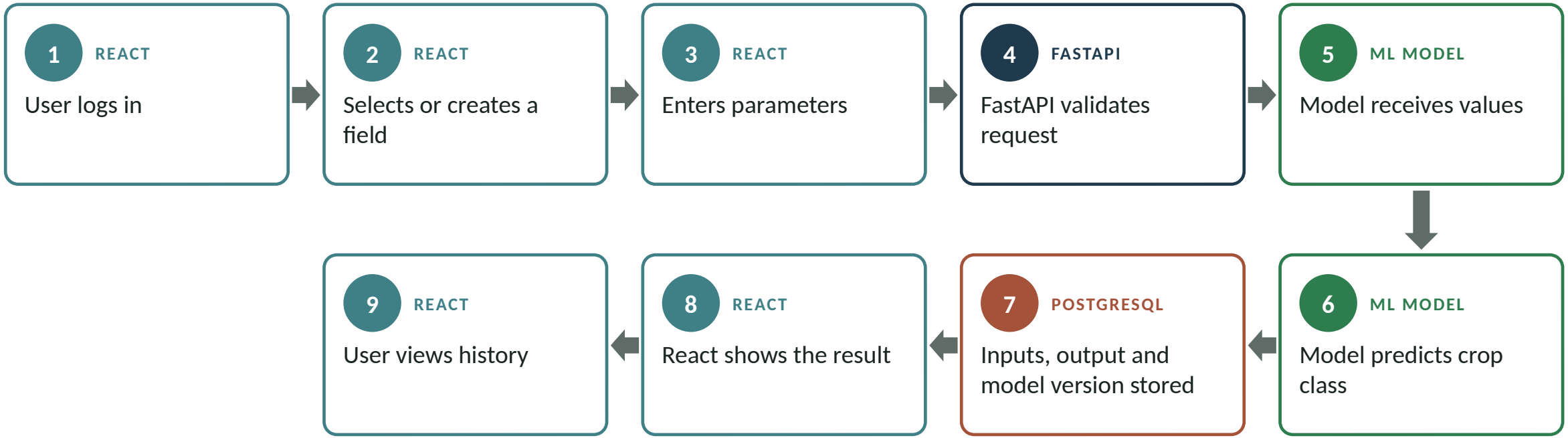
Design choice

The model loads inside FastAPI — no separate ML server needed at first

END-TO-END WORKFLOW

From Input to Crop Recommendation

Proposed user workflow



Note: the prediction comes from the model; saving it with the model version is a separate backend step for traceability.

ML METHOD 1

Input Features and Supervised Learning

Learning from labelled examples

HOW IT WORKS

- Training data: features X (soil and weather) with a known crop label y
- The model learns the mapping X → crop label
- For new inputs it predicts one supported crop class
- Units and valid ranges must match the chosen dataset

Illustrative feature set (subject to the selected dataset)

Feature	Meaning
N	Soil nitrogen measurement
P	Soil phosphorus measurement
K	Soil potassium measurement
Temperature	Environmental temperature
Humidity	Relative humidity
pH	Soil acidity / alkalinity
Rainfall	Rainfall measurement

ML METHOD 2

Candidate Algorithms and Training Pipeline

Three lightweight, explainable classifiers to compare

PIPELINE

- Validate and clean the labelled dataset
- Split into training, validation and test sets
- Train Decision Tree, Random Forest and KNN
- Select the model from measured results, then integrate with FastAPI

Decision Tree
Learns a hierarchy of feature-based rules

Random Forest
Combines many trees by majority vote

K-Nearest Neighbors
Uses nearby labelled examples; needs feature scaling

IMPLEMENTATION

How React Communicates With Python

Proposed REST API — a design, not running code

PROPOSED ENDPOINTS

- GET /api/v1/crops
- POST /api/v1/recommendations
- GET /api/v1/recommendations (history)
- POST and GET /api/v1/fields
- User identity always comes from authentication

POST /api/v1/recommendations · illustrative body

```
{
  "field_id": 5,
  "nitrogen": 70,
  "phosphorus": 40,
  "potassium": 50,
  "temperature": 27,
  "humidity": 75,
  "ph": 6.5,
  "rainfall": 180
}
```

Values are illustrative, not validated agricultural measurements; units and bounds will come from the dataset.

DATABASE DESIGN

What PostgreSQL Will Store

Conceptual entities — final schema designed during implementation

USER

email · password hash

FIELD

user · name · location

RECOMMENDATION

field · crop · model version

RECOMMENDATION_INPUT

feature · value · unit

CROP

name

CROP_INFORMATION

crop · description · source

MODEL_VERSION

algorithm · version · date

KEY RELATIONSHIPS

- One user manages many fields
- One field has many recommendations
- Each recommendation records its model version
- Each recommendation stores its inputs and crop class
- Application records are not the training dataset

EVALUATION PLAN

How We Will Evaluate the System

Planned methodology — no experiments have been run yet

WHAT WE WILL MEASURE

- Accuracy
- Precision and recall
- Macro-F1 (rare crops count equally)
- Confusion matrix
- Per-crop performance

ALSO TESTED

Login, missing or non-numeric input, record ownership, history storage and missing model file (Pytest).

Model comparison

RESULTS PENDING

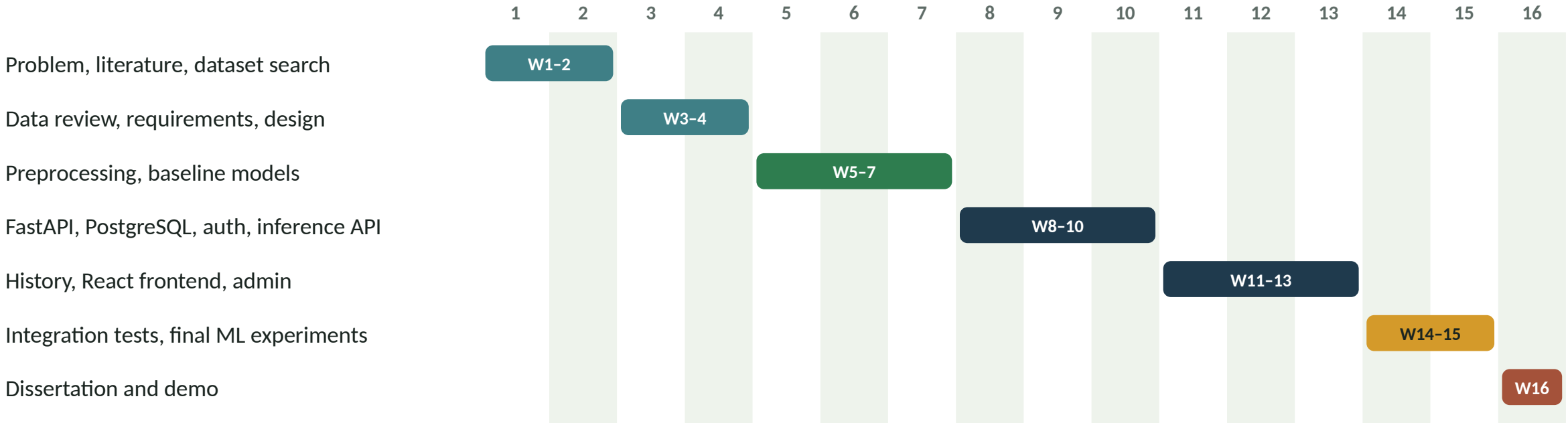
Model	Accuracy	Macro-F1
Decision Tree	TBD	TBD
Random Forest	TBD	TBD
KNN	TBD	TBD

All values will come from real evaluation code on a held-out test set. A high score on one dataset does not prove suitability for every region or season.

PROJECT PLAN

Implementation Timeline and Milestones

Proposed 16-week work plan



2 Dec 2026
Mid-semester report

20 Jan 2027
Final report

Week 16
Final demonstration

CONCLUSION

Expected Outcomes and Limitations

What we expect to deliver — and what we will not claim

Expected outcomes

- A working crop-recommendation web prototype
- A measured comparison of three classifiers
- Traceable history with model versions
- Clear communication of limitations

Limitations

- Depends on the chosen dataset and its crops
- Regions, seasons and practices may differ
- Predictions can be wrong
- No yield guarantee; not a substitute for agronomists

"Model-based crop recommendation for informational decision support; consult qualified agricultural guidance before making cultivation decisions."

THANK YOU

Thank You — Questions?

We welcome feedback from our supervisor and reviewers

KEY TAKEAWAYS

1. Soil and weather measurements in → crop class out
2. Three classifiers compared on real evidence
3. Every recommendation stored with its model version
4. Decision support, not a guarantee

PRESENTED BY

[Student Name]
[BITS ID]
[Email address]

SUPERVISOR

[Supervisor Name]
[Organisation]

CropAssist is a proposed project. All examples, screens and values in this presentation are illustrative.